Software Architecture and Design Specification

Project: Railway Reservation System (RRS)

Team: Jackfruit Phase-1

Version 1.2

TEAM 13

Authors:
Yellapantula Venkata Pranav — PES2UG24CS616
Narendra Kathare — PES2UG25CS814
Yashash C — PES2UG24CS611

Date: 14-09-2026
Status: Draft

Revision History

Version	Date	Author	Change Summary
1.0	14-09-2026	Yellapantula Venkata Pranav (PES2UG24CS616)	Drafted SAD sections 1–3.2, 3.5–3.9 from SRS; drew UML Component Diagram (Section 3.3)
1.1	14-09-2026	Narendra Kathare (PES2UG25CS814)	Drafted Section 4 (Design Overview, API Design, Error Handling, UX); drew UML Sequence Diagram 1 — Search, Book & Pay (Section 4.2)
1.2	14-09-2026	Yashash C (PES2UG24CS611)	Drafted Section 3.4 Component Descriptions and Section 5 Appendices; drew UML Sequence Diagram 2 — Cancellation, Refund & Waitlist Promotion (Section 4.2)

Approvals

Role	Name	Signature/Date
Course Coordinator		

1. Introduction

1.1 Purpose

This document specifies the software architecture and design of the Railway Reservation System (RRS), a self-contained academic web application for booking, tracking, and managing train reservations. It is intended to guide implementation, review, and evaluation, building on the RRS Software Requirements Specification (SRS) v1.0.

1.2 Scope

Covers passenger-facing services — searching trains, checking seat/berth availability, booking, payment (simulated), PNR tracking, and cancellation/refund — as well as administrative functions for managing trains, schedules, fares, and reports. Excludes integration with real railway infrastructure (e.g. IRCTC) and real payment gateways, which are represented by a mock payment module.

1.3 Audience

Developers, QA engineers, system integrators, and assessment evaluators.

1.4 Definitions

RRS (Railway Reservation System), PNR (Passenger Name Record), RAC (Reservation Against Cancellation), UI (User Interface), API (Application Programming Interface), DB (Database), TLS (Transport Layer Security), CLI (Command-Line Interface), ADR (Architecture Decision Record).

2. Document Overview

2.1 How to use this document

Provides the architectural and design deliverables for RRS: component (UML) diagrams, sequence diagrams, API design, security architecture, and traceability back to SRS requirements.

2.2 Related Documents

RRS Software Requirements Specification (SRS) v1.0, dated 05-09-2026.

3. Architecture

3.1 Goals & Constraints

Goals:

- Overall booking transaction response ≤ 5 seconds for 90% of transactions under normal load (RRS-NF-001).
- 99% availability during scheduled demo/evaluation periods (RRS-NF-002).
- Protection of sensitive user and payment data; no plaintext storage of passwords or card-like data (RRS-NF-003).

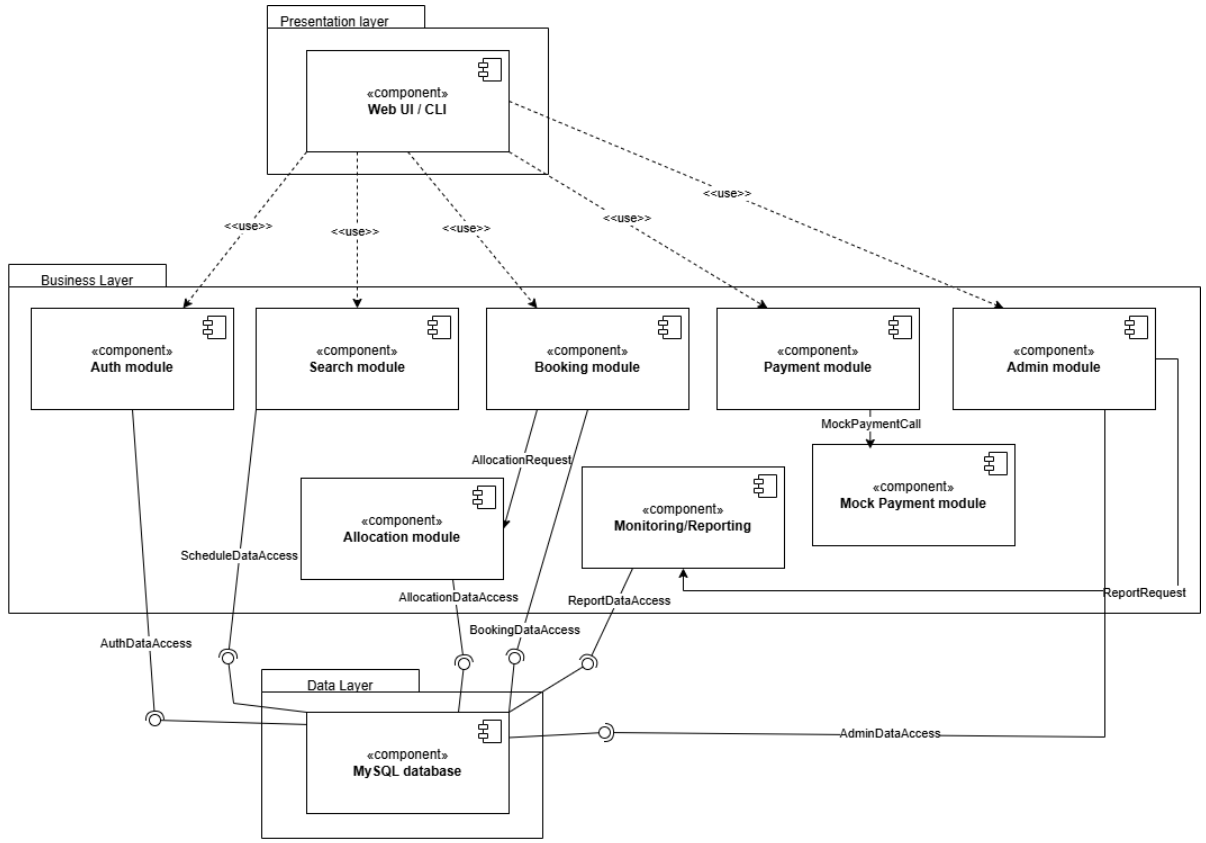
Constraints:

- Booking, allocation, storage, and presentation must remain separate, swappable layers behind defined interfaces.
- Payment is handled only by a simulated/mock module — no real payment gateway is integrated for this course project.
- Only synthetic/sample train, schedule, and passenger data is used — no real personal data.
- Must run comfortably on a standard student laptop/development server.

3.2 Stakeholders & Concerns

Stakeholder	**Concerns**
Passenger	Short, clear booking flow; visibility into booking/PNR status
Administrator	Dashboard with clear, actionable train/schedule/booking information
Evaluator / Instructor	Clean separation between booking, allocation, storage, and presentation layers

3.3 Component (UML) Diagram



3.4 Component Descriptions

Component	Description
Web UI / CLI	Presentation layer — search/book/pay/track/cancel menu; accessible, high-contrast status indicators
Backend API	Flask HTTP/JSON layer routing requests to the appropriate module
Auth Module	Registration, login, session/idle timeout, account lockout after repeated failures
Search Module	Finds trains by source, destination, and date against schedule data
Booking Module	Creates provisional bookings, generates PNR, handles cancellation and refund calculation
Payment Module	Routes payment requests to the Mock Payment Module and confirms bookings on success
Mock Payment Module	Simulates a payment gateway call, returns Success/Failure; stores only a transaction reference, never card-like data
Allocation Module	Assigns Confirmed/RAC/Waitlisted status by capacity; promotes waitlisted/RAC bookings on cancellation
Admin Module	Add/update/remove trains and schedules; role-restricted at the server, not just the UI
Monitoring / Reporting	Booking, occupancy, and revenue reports; audit log of administrative actions
MySQL Database	Repository-style storage for users, trains, schedules, bookings, and payment records

3.5 Chosen Architecture Pattern and Rationale

Layered architecture was chosen to give a clear separation between presentation, API, business modules (auth, search, booking, payment, allocation, admin), and data storage — directly matching the SRS constraint that these layers stay separate and swappable. A microservices approach was considered and rejected as unnecessarily complex for the scale and evaluation timeline of this course project.

3.6 Technology Stack & Data Stores

- Backend: Python 3.x with Flask, serving HTTP(S)/JSON APIs
- Database: MySQL, accessed through a repository-style interface
- Communication: HTTPS / TLS 1.2+ enforced for all client–server traffic in deployment
- Payment: mock/simulated module only — no real payment gateway integration
- Client: any modern web browser, or a terminal for the CLI build

3.7 Risks & Mitigations

Risks	Mitigation
Mock payment times out or fails, leaving a seat held indefinitely	Provisional booking and held seat are released back to availability on failure/timeout (RRS-F-031)

Risks	Mitigation
Database unavailable during a demo/evaluation session	Connection retries and backups; layered design keeps the DB behind a repository interface for easier failover
Race condition on the last available seat/berth during concurrent bookings	Allocation Module owns seat-count updates and assigns Confirmed/RAC/Waitlisted status at booking time (RRS-F-050)
Card-like data accidentally logged or persisted	Mock Payment Module stores only a transaction reference and status, never raw card fields (PRJ-SR-005)

3.8 Traceability to Requirements

Req ID	Requirement (short)	Module
RRS-F-001	Register / authenticate user	Auth Module
RRS-F-010	Search trains by route/date	Search Module
RRS-F-021	Create provisional booking	Booking Module
RRS-F-030	Confirm booking after payment	Payment Module
RRS-F-040	Generate PNR / status lookup	Booking Module
RRS-F-041	Cancel booking, release seat	Booking Module
RRS-F-050	Auto seat status assignment	Allocation Module
RRS-F-051	Waitlist/RAC promotion	Allocation Module
RRS-F-060	Admin train/schedule management	Admin Module
RRS-NF-001	Response time target	Web UI / Backend API
PRJ-SR-002	Password hashing	Auth Module
PRJ-SR-003	Server-side RBAC	Admin Module

3.9 Security Architecture

Threat Modeling (STRIDE), mapped to the SRS security requirements (PRJ-SR-001 to PRJ-SR-006):

STRIDE Category	Mitigation
Spoofing	Authenticated login; session tokens expire after a configured idle timeout (PRJ-SR-004)
Tampering	Role-based access control enforced at the server for admin-only endpoints, not just hidden in the UI (PRJ-SR-003)
Information Disclosure	TLS 1.2+ mandatory for all network connections; passwords hashed, never stored in plaintext (PRJ-SR-001, PRJ-SR-002)
Denial of Service	Account lockout after repeated failed login attempts (RRS-F-003); provisional bookings auto-release on timeout
Repudiation	All administrative actions (schedule changes, refunds) logged with user identity and timestamp (PRJ-SR-006)

STRIDE Category	Mitigation
Elevation of Privilege	Server-side RBAC; Mock Payment Module never persists full card-like details (PRJ-SR-003, PRJ-SR-005)

4. Design

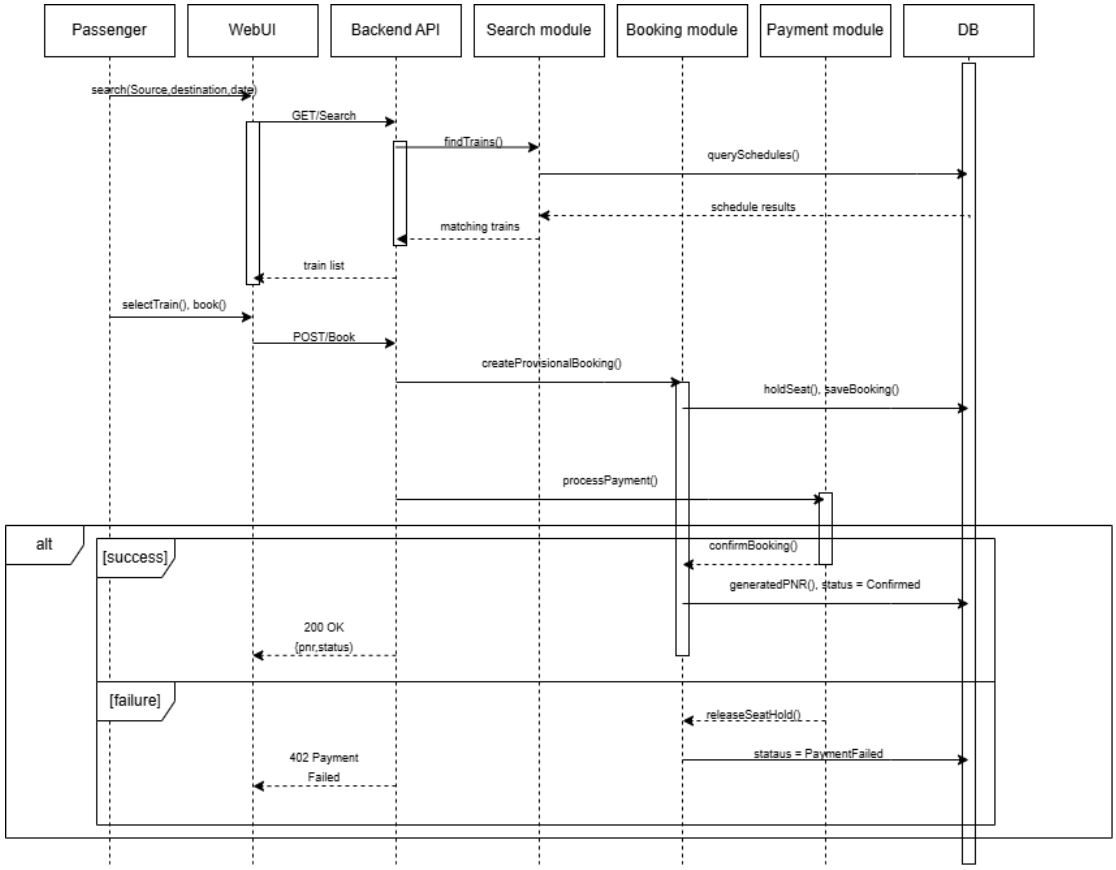
4.1 Design Overview

RRS is designed as layered modules — presentation, API, business logic (auth, search, booking, payment, allocation, admin), and data storage — for clear separation of concerns and easier maintenance and testing.

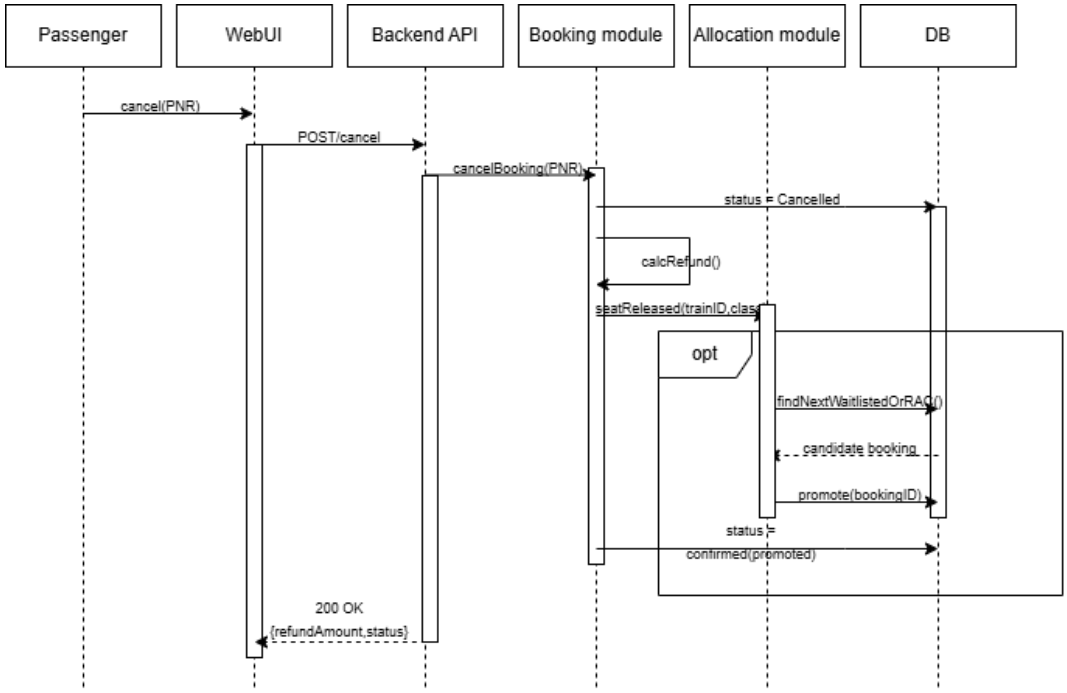
4.2 UML Sequence Diagrams

Two flows specific to this project are detailed below.

Sequence Diagram 1 — Search, Book & Pay (RRS-F-010, 020, 021, 030, 040)



Sequence Diagram 2 — Cancellation, Refund & Waitlist/RAC Promotion (RRS-F-041, 042, 051)



4.3 API Design

Interface definitions for two core endpoints:

POST /book

- Request: {trainId, class, passengerDetails}
- Response: {bookingId, status, seatInfo}
- Errors: 400 invalid/missing fields (RRS-F-022); 409 no seats available

POST /pay

- Request: {bookingId, mockCardRef}
- Response: {status, pnr}
- Errors: 402 payment failed/timed out — booking and seat hold are released (RRS-F-031)

4.4 Error Handling, Logging & Monitoring

Standardized error messages across the API. No sensitive information (including card-like data) is written to logs (PRJ-SR-005). Booking and audit logs are timestamped and retained for the evaluation period (RRS-NF-004); all administrative actions are logged with user identity and timestamp (PRJ-SR-006). Monitoring covers transaction failure rate and system uptime, reported through the Monitoring/Reporting module.

4.5 UX Design

A simple web form (or CLI) with a clear menu: search, book, pay, track PNR, cancel. Accessibility follows the SRS usability requirement (RRS-NF-005) — legible fonts, clear labels, and high-contrast, readable status indicators for Confirmed / RAC / Waitlisted / Cancelled bookings.

4.6 Open Issues & Next Steps

- Integration with a real payment gateway beyond the current mock module
- Mobile app / responsive client in addition to the web form and CLI

- Extending reporting with predictive occupancy analytics

5. Appendices

5.1 Glossary

RRS, PNR, RAC, UI, API, DB, TLS, CLI, ADR — see Section 1.4.

5.2 References

IEEE 42010, OWASP, NIST SP 800-160; RRS Software Requirements Specification v1.0.

5.3 Tools

PlantUML, draw.io, Swagger.